

**FRESHROUTE - A SMART FRUIT SUPPLY CHAIN
SYSTEM TO REDUCE WASTE AND ELIMINATE
MIDDLEMEN IN SRI LANKA**

Draft Thesis Report

Samarakoon A. D.

(IT22124494)

**B.Sc. (Hons) Degree in Information Technology Specializing in Software
Engineering**

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other University or Institute of Higher Learning. To the best of my knowledge and belief, it does not contain any material previously published or written by another person except where acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation, in whole or in part, in print, electronic, or other medium. I retain the right to use this content in whole or part in future works such as articles or books.

Name	Student ID	Signature	Date
Samarakoon A. D.	IT22124494	[Insert Signature]	4/16/2026

The supervisor/s should certify the dissertation with the following declaration.

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Supervisor	Signature	Date
Mrs. Karthiga Rajendran	[Insert Signature]	[Insert Date]
Ms. Poojani Gunathilake	[Insert Signature]	[Insert Date]

ABSTRACT

Sri Lanka's fruit supply chain experiences persistent post-harvest losses, price instability, and information asymmetry that weaken the bargaining position of smallholder farmers. This thesis presents FreshRoute, an AI-driven demand forecasting and alert system designed to support harvest planning, reduce avoidable waste, and improve market responsiveness in the Sri Lankan fruit sector. The proposed component forms the forecasting intelligence layer of a broader smart fruit supply chain platform. It collects historical market prices, weather observations, weather forecasts, and seasonal yield indicators; preprocesses these heterogeneous data streams; and trains a multivariate Long Short-Term Memory (LSTM) time-series model to forecast short-term demand and price movements for selected fruit categories. To ensure practical use in rural environments, the model outputs are translated into farmer-facing advisories such as expected demand level, price trend, confidence score, and recommended harvest timing. These advisories are delivered through an offline-capable mobile dashboard and a multi-channel notification workflow using SMS and Firebase push notifications. The system architecture uses FastAPI-based prediction services, PostgreSQL for structured storage, Redis for low-latency forecast retrieval, REST APIs for integration, and TLS-protected HTTPS communication for secure data exchange. The evaluation framework combines model accuracy metrics, API performance testing, notification delivery validation, mobile usability testing, and user acceptance testing. While final field measurements must be inserted after pilot completion, the draft evaluation design demonstrates how FreshRoute can be assessed against baseline methods such as SARIMA and Prophet. The research contributes a context-sensitive software engineering solution that connects advanced forecasting with accessible decision support for farmers facing low connectivity, limited digital literacy, and volatile market conditions.

Keywords: Demand Forecasting, LSTM, Smart Agriculture, Fruit Supply Chain, Post-Harvest Loss

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Ms. Karthiga Rajendran, for her continuous guidance, constructive feedback, and academic support throughout the research and development of the FreshRoute demand forecasting component. Her expertise and encouragement greatly contributed to the refinement of the research direction, system architecture, and evaluation design.

I also extend my sincere thanks to my co-supervisor, Ms. Poojani Gunathilaka, for her valuable suggestions and support in improving the practical relevance and technical quality of the proposed solution. I am grateful to the academic and non-academic staff of the Faculty of Computing, Sri Lanka Institute of Information Technology, for providing the learning environment, facilities, and resources required to complete this work.

I appreciate the support of my project team members for their collaboration in developing the broader FreshRoute ecosystem, including the marketplace, logistics, and quality verification modules that complement the forecasting system. I also acknowledge the contribution of agricultural stakeholders, publicly available data sources, and prior researchers whose work helped frame the problem of post-harvest loss and market inefficiency in Sri Lanka.

Finally, I am deeply thankful to my family and friends for their patience, motivation, and confidence in me throughout the final year research journey.

Table of Contents

DECLARATION	2
ABSTRACT	3
ACKNOWLEDGEMENT	4
LIST OF TABLES	i
LIST OF FIGURES	ii
LIST OF ABBREVIATIONS	iii
1 INTRODUCTION	1
1.1 Background Study and Literature Review	1
1.1.1 Background study	1
1.1.2 Information asymmetry and middleman dependence	2
1.1.3 Literature review on agricultural forecasting	3
1.1.4 Digital agriculture and rural delivery channels	4
1.1.5 FreshRoute as an integrated supply chain platform	4
1.2 Research Gap	6
1.2.1 Limitations of existing systems	6
1.2.2 Gap in research and practice	7
1.2.3 Research questions	7
1.3 Research Problem	8
1.4 Research Objectives	9
1.4.1 Main objective	9
1.4.2 Specific objectives	10
1.4.3 Business and social objectives	11
1.5 Scope of the Thesis	11

1.6 Organization of the Thesis	11
2 METHODOLOGY.....	13
2.1 Methodology	13
2.1.1 Research and development approach	13
2.1.2 Requirement gathering and stakeholder analysis	13
2.1.3 Overall system architecture.....	17
2.1.4 Data collection methodology	18
2.1.5 Data preprocessing and feature engineering	20
2.1.6 LSTM model development	22
2.1.7 Baseline models and evaluation metrics	23
2.1.8 Backend and API design	24
2.1.9 PostgreSQL database design	24
2.1.10 Redis caching and performance considerations	25
2.1.11 Mobile application design and offline capability	26
2.1.12 Alert generation workflow	26
2.1.13 Security and privacy design	27
2.1.14 Integration with broader FreshRoute modules	28
2.1.15 Tools and technologies.....	29
2.2 Commercialization Aspect of the Product	30
2.2.1 Value proposition	30
2.2.2 Business model	31
2.2.3 Market entry and rollout strategy	32
2.2.4 Sustainability and social impact.....	32
2.3 Testing and Implementation.....	33

2.3.1 Implementation phases.....	33
2.3.2 Model training implementation.....	33
2.3.3 API implementation	33
2.3.4 Mobile implementation	34
2.3.5 Testing strategy	34
2.3.6 Playwright and end-to-end web testing context.....	36
2.3.7 Deployment plan	36
2.3.8 Feasibility and risk management.....	37
3 RESULTS AND DISCUSSION	38
3.1 Results	38
3.1.1 Overview of evaluation.....	38
3.1.2 Data preparation results.....	38
3.1.3 Forecasting model results.....	40
3.1.4 Backend and API results	41
3.1.5 Alert and notification results.....	41
3.1.6 Mobile usability results	42
3.1.7 Integrated platform results	42
3.2 Research Findings	43
3.3 Discussion	43
3.3.1 Technical interpretation	43
3.3.2 Practical interpretation for farmers	44
3.3.3 Discussion of security and trust	44
3.3.4 Limitations	44
3.3.5 Future enhancement	46

4 CONCLUSION47

 4.1 Conclusion47

 4.2 Recommendations48

 4.3 Future Work48

REFERENCES.....50

GLOSSARY52

APPENDICES54

 Appendix A: A Price List at Dambulla Dedicated Economic Center54

 Appendix B: Detailed Test Cases55

**FRESHROUTE - A SMART FRUIT SUPPLY CHAIN
SYSTEM TO REDUCE WASTE AND ELIMINATE
MIDDLEMEN IN SRI LANKA**

Draft Thesis Report

Samarakoon A. D.

(IT22124494)

**B.Sc. (Hons) Degree in Information Technology Specializing in Software
Engineering**

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

LIST OF TABLES

<i>Table 1. 1: Problems with existing agricultural information systems</i>	<i>6</i>
<i>Table 1. 2: Research questions and expected evidence</i>	<i>8</i>
<i>Table 2. 1: Functional requirements of FreshRoute forecasting component</i>	<i>14</i>
<i>Table 2. 2: Non-functional requirements of the proposed system</i>	<i>15</i>
<i>Table 2. 3: Data sources used by the forecasting module</i>	<i>19</i>
<i>Table 2. 4: Data preprocessing and feature engineering operations</i>	<i>21</i>
<i>Table 2. 5: Tools and technologies used in implementation</i>	<i>29</i>
<i>Table 2. 6: Commercialization strategy summary</i>	<i>31</i>
<i>Table 2. 7: Test plan for the forecasting and alert system</i>	<i>35</i>
<i>Table 3. 1: Evaluation metrics and interpretation</i>	<i>39</i>
<i>Table G. 1: Glossary of key terms</i>	<i>52</i>

LIST OF FIGURES

<i>Figure 1. 1: Sri Lanka fruit supply chain problem context.....</i>	<i>2</i>
<i>Figure 1. 2: FreshRoute closed-loop decision concept</i>	<i>5</i>
<i>Figure 2. 2: High-level architecture of the FreshRoute demand forecasting system</i>	<i>17</i>
<i>Figure 2. 3: System overview and data flow</i>	<i>18</i>
<i>Figure 2. 4: LSTM model development structure</i>	<i>23</i>
<i>Figure 2. 5: Mobile forecast design UI.....</i>	<i>26</i>
<i>Figure 3. 1: Forecast Model Validation</i>	<i>40</i>
<i>Figure 3. 4: User acceptance feedback Collection</i>	<i>42</i>
<i>Figure 3. 5: Future Works with pro Version.....</i>	<i>46</i>

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
AWS	Amazon Web Services
CA	Certificate Authority
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DB	Database
FCM	Firebase Cloud Messaging
GCP	Google Cloud Platform
GPS	Global Positioning System
HARTI	Hector Kobbekaduwa Agrarian Research and Training Institute
HTTPS	Hypertext Transfer Protocol Secure
IoT	Internet of Things
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MSE	Mean Squared Error

MVP	Minimum Viable Product
NIPHM	National Institute of Post Harvest Management
REST	Representational State Transfer
RMSE	Root Mean Square Error
SARIMA	Seasonal Autoregressive Integrated Moving Average
SDLC	Software Development Life Cycle
SLIIT	Sri Lanka Institute of Information Technology
SMS	Short Message Service
SQL	Structured Query Language
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UAT	User Acceptance Testing
UI	User Interface
UX	User Experience
WAPE	Weighted Absolute Percentage Error
WBS	Work Breakdown Structure

1 INTRODUCTION

1.1 Background Study and Literature Review

1.1.1 Background study

Sri Lanka has strong horticultural potential, but a substantial proportion of fruit value is lost before produce reaches consumers. The fruit supply chain contains many operational points where value can be reduced: harvest planning, farm-gate selling, aggregation, transport, wholesale distribution, retail handling, and complaint resolution. Studies cited in the FreshRoute research context report post-harvest fruit losses of approximately 30-40% nationally, while commodity-level analysis of banana supply chains has identified measurable losses across harvesting, transportation, wholesale, and retail stages [1], [2]. Official reports also indicate that fruit losses during transportation are strongly associated with inappropriate handling and packaging practices [3], [4].

For smallholder farmers, such losses are not merely technical inefficiencies. They directly affect household income, production planning, and bargaining power. When farmers harvest without knowing expected demand or market price movement, they risk sending fruit to market during oversupply periods. Oversupply depresses prices, while delays and poor handling increase spoilage. The result is a cycle in which producers bear risk but intermediaries often retain stronger control over market information. This problem became more severe during the economic instability experienced by Sri Lanka, because increased fertilizer, transport, and input costs reduced farmer margins and made each wasted kilogram economically more damaging.

The central premise of this thesis is that reducing waste requires more than improving transport or packaging after harvest. It requires earlier, data-informed decision-making. Farmers need practical forecasts that indicate whether demand is expected to rise, fall, or remain stable; whether a harvest should be accelerated or delayed; and how strongly a recommendation should be trusted. Therefore, FreshRoute is designed as a smart supply

chain system in which the demand forecasting module serves as the intelligence layer that supports farmer planning before produce enters the market.

Crop	Operations					Total loss in farm operations	Storage Channels					Total loss in storage	Overall Total Loss
	Harvesting	Collection	Sorting/Grading	Packaging	Transport		Farm	Godown/cold store	Whole-saler	Retailer	Processing unit		
Apple	4.33 ±0.35	0.29 ±0.22	3.94 ±0.44	0.11 ±0.11	0.42 ±0.19	9.08[#] ±0.30	0.02 ±0.00	0.13 ±0.02	0.57 ±0.08	0.34 ±0.29	0.25 ±0.11	1.31 ±0.11	10.39[*] ±0.24 (12.26)
Banana	1.62 ±0.35	0.26 ±0.14	2.06 [*] ±0.37	0.19 ±0.31	1.91 ±0.40	6.04 ±0.33	0.03 ±0.01	0.08 [#] ±0.04	1.16 ±0.35	0.45 ±0.07	0.00 [*] ±0.00	1.72 ±0.17	7.76 ±0.29 (6.60)
Citrus	1.68 ±0.48	0.33 ±0.10	3.71 ±0.59	0.18 ±0.13	1.65 ±0.24	7.55[*] ±0.38	0.04 ±0.01	0.02 [*] ±0.01	0.91 ±0.13	1.12 ±0.20	0.06 ±0.07	2.14[*] ±0.11	9.69[*] ±0.29 (6.38)
Grapes	1.77 ±0.2	0.30 ±0.05	3.36 ±0.42	0.10 ±0.06	0.98 ±0.18	6.52 ±0.26	0.01 [#] ±0.00		0.78 ±0.14	1.24 ±0.17	0.09 ±0.03	2.11 ±0.15	8.63 ±0.22 (8.30)
Guava	5.33 ±1.78	0.31 ±0.28	4.95 ±1.86	0.09 ±0.20	1.21 [#] ±0.64	11.90 ±1.24	0.23 ±0.06		1.62 ±0.52	2.08 ±0.73	0.04 [#] ±0.03	3.98± 0.55	15.88 ±1.11 (18.04)
Mango	2.09 ±0.71	0.30 ±0.44	3.26 ±0.77	0.23 ±0.07	1.04 ±0.14	6.92[#] ±0.54	0.11 ±0.01	0.01 ±0.01	0.69 ±0.39	1.18 ±0.42	0.25 ±0.09	2.24 ±0.28	9.16[#] ±0.50 (12.74)
Papaya	0.98 ±0.45	0.42 ±0.15	1.46 ±0.47	0.34 ±0.06	0.92 ±0.39	4.12 ±0.37	0.05 ±0.02	0.01 [*] ±0.00	0.79 ±0.17	1.71 ±0.25	0.03 [*] ±0.01	2.58± 0.12	6.70± 0.26 (7.34)
Sapota	2.53 ±1.18	0.35 ±0.07	2.55 ±0.56	0.28 ±0.17	1.70 ±0.48	7.41[*] ±0.67	0.01 ±0.01	0.25 [*] ±0.05	0.89 ±0.20	1.13 ±0.13	0.03 [*] ±0.02	2.31[*] ±0.09	9.73[*] ±0.46 (5.77)

^{*} Estimated losses are significantly higher in comparison to losses observed in 2005-07(previous study).
[#] Estimated losses are significantly lower in comparison to losses observed in 2005-07(previous study).
 Figures in parenthesis represent the losses of previous study 2005-07.

Figure 1. 1: Sri Lanka fruit supply chain problem context

1.1.2 Information asymmetry and middleman dependence

Information asymmetry is one of the most significant structural causes of inefficiency in traditional agricultural markets. Farmers often know their production capacity and crop condition, but they do not have reliable visibility into buyer demand, regional price movements, wholesale activity, or short-term weather effects on supply. Intermediaries, collectors, and commission agents typically possess stronger knowledge of market conditions and can use this advantage during price negotiation. As a result, farmers may accept low farm-gate prices even when downstream demand is stronger than they perceive.

The proposal identifies this information gap as the primary reason farmers make harvest decisions using memory, informal advice, and delayed price reports. Existing digital agriculture initiatives in Sri Lanka, such as mobile applications and advisory portals, have improved access to general information. However, many systems still provide static or historical information rather than predictive, actionable recommendations. A farmer who

only receives last week's price still cannot determine whether tomorrow's price is likely to rise or fall. Therefore, the need is not only for digitization but for anticipatory intelligence delivered in a form that rural users can act upon.

FreshRoute responds to this gap by treating forecasts as decision support rather than as technical output. The system converts model predictions into messages such as DELAY_HARVEST, SELL_SOON, or NO_CHANGE. These advisories are designed to reduce cognitive burden, particularly for farmers who may not be comfortable interpreting statistical charts. The decision-support layer therefore acts as a bridge between machine learning and practical agricultural behavior.

1.1.3 Literature review on agricultural forecasting

Demand and price forecasting have long been studied using statistical models such as ARIMA and SARIMA. These models are useful when time-series data exhibit stable trends and seasonal patterns; however, agricultural markets are often influenced by nonlinear interactions between weather, production cycles, holidays, transport conditions, and sudden demand changes. SARIMA-based agricultural studies demonstrate the value of time-series modeling, but they also reveal limitations when datasets become multivariate or highly nonlinear [6].

More recent work has shown that neural networks, especially Long Short-Term Memory (LSTM) models, can capture temporal dependencies that conventional statistical models may fail to represent. LSTM networks are a type of recurrent neural network designed to retain useful information over sequences through gated memory cells. This makes them appropriate for forecasting tasks in which current demand depends not only on the immediate previous day, but also on longer patterns such as rainfall cycles, price momentum, festival seasons, and delayed supply reactions. Research on agricultural sales forecasting has shown that seasonal LSTM approaches can outperform traditional models in several contexts [8]. Comparative studies between ARIMA and LSTM also indicate that recurrent neural networks are effective for complex time-series forecasting when sufficient data and careful validation are available [9].

Prophet, introduced by Taylor and Letham, provides another useful baseline because it decomposes time series into trend, seasonality, and holiday components while remaining interpretable and relatively robust to missing observations [7]. In this thesis, Prophet and SARIMA are retained as baseline methods rather than being discarded. This is important because a production system should not rely blindly on a single model. If the LSTM model performs poorly for a crop with sparse data, simpler models may provide a fallback forecast or at least a benchmark for interpretation.

1.1.4 Digital agriculture and rural delivery channels

The effectiveness of a forecasting model depends on whether its outputs reach the intended users at the correct time. In rural agricultural environments, connectivity constraints, device diversity, digital literacy, and language preferences are as important as algorithmic accuracy. Studies of SMS-based agricultural information systems show that mobile delivery can influence farmer awareness and decision-making when information is timely, trusted, and easy to understand [12]. For this reason, FreshRoute combines an Android mobile dashboard with SMS and push notifications rather than depending solely on a smartphone application.

The offline-capable dashboard addresses the common situation where farmers temporarily lack reliable mobile data access. Forecasts are cached locally, marked with the last synchronization time, and updated when connectivity returns. Push notifications are delivered through Firebase Cloud Messaging for smartphone users, while SMS alerts are delivered through Twilio or a local telecommunications gateway for feature-phone users. This dual-channel design supports inclusiveness by acknowledging that the most economically vulnerable farmers may not own high-end smartphones or maintain continuous data connectivity.

1.1.5 FreshRoute as an integrated supply chain platform

Although this thesis focuses on the AI-driven forecasting and alert module, FreshRoute is conceptualized as a broader smart fruit supply chain platform. The integrated research paper describes four mutually reinforcing modules: demand forecasting, a permissioned

marketplace, perishability-aware logistics optimization, and AI-based quality verification. The forecasting module influences listing timing and expected supply, the marketplace records orders and commitments, the logistics module plans freshness-aware transport, and the quality module supports evidence-based complaint validation. This closed-loop design is important because isolated tools often fail to change supply chain outcomes when they are not connected to actual trading and delivery workflows.

Within this integrated environment, the demand forecasting component has two roles. First, it provides farmers with actionable intelligence for harvest planning. Second, it creates upstream data that can improve the behavior of downstream modules. For example, if forecasts indicate high demand for bananas in a particular region, farmers can list produce earlier, buyers can make commitments sooner, and logistics providers can plan vehicle capacity more efficiently. Thus, forecasting is not an analytical endpoint; it is a coordination mechanism for the fruit supply chain.

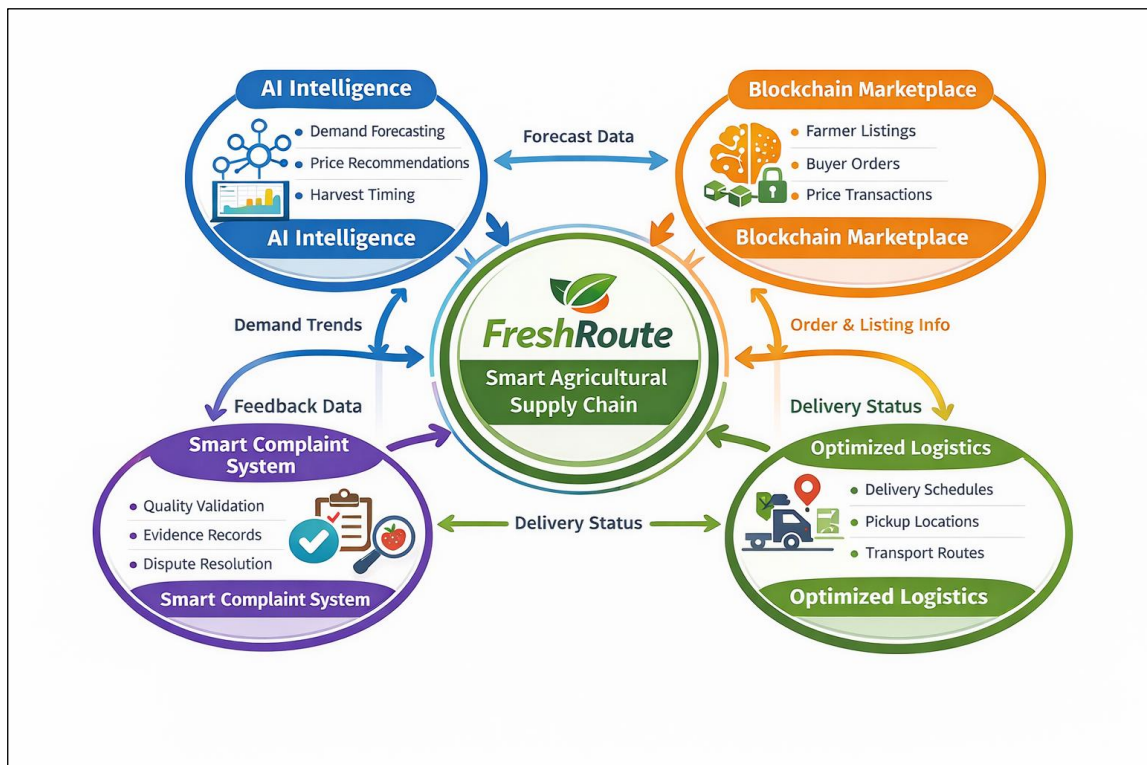


Figure 1. 2: FreshRoute closed-loop decision concept

1.2 Research Gap

1.2.1 Limitations of existing systems

The proposal identifies several limitations in existing agricultural digital systems. First, many systems are descriptive rather than predictive. They provide historical price lists, general advisories, or static weather information, but they do not forecast future demand conditions. Second, they lack personalization. Farmers need recommendations based on crop type, location, season, and planned harvest window, rather than broad messages that may not apply to their field situation. Third, many tools are not integrated with market access, logistics, or alerts, which limits their ability to support end-to-end decision-making.

Another gap concerns accessibility. Many systems assume consistent internet access and smartphone ownership. However, smallholder farmers may rely on feature phones or experience intermittent connectivity. Without offline functionality and SMS fallback, even technically strong systems may fail at the last mile. Finally, there is limited evidence of deployed Sri Lankan fruit forecasting systems that combine LSTM-based multivariate forecasting, mobile visualizations, SMS alerts, and integration-ready APIs for a broader supply chain platform.

Table 1. 1: Problems with existing agricultural information systems

Category	Limitation	Practical Effect on Farmers
Static information delivery	Systems focus on historical or current prices rather than future demand.	Farmers continue to make harvest decisions using delayed information.
Limited personalization	Advisories are often generic and not crop, district, or season specific.	Recommendations may not match the farmer's actual harvest context.

Weak integration	Forecasting, trading, logistics, and dispute handling are treated separately.	Digital tools provide partial support but do not reduce end-to-end supply chain friction.
Connectivity dependency	Many tools assume stable internet and smartphone use.	Rural farmers with feature phones or poor connectivity are excluded.
Lack of advanced forecasting	Local systems rarely deploy multivariate LSTM-based live forecasts.	Farmers lack proactive intelligence to anticipate price and demand volatility.

1.2.2 Gap in research and practice

The academic literature demonstrates that LSTM and other machine learning models can forecast agricultural prices and sales volumes; however, many studies remain proof-of-concept experiments. They often emphasize model accuracy while giving less attention to operational deployment, user interface design, alert reliability, or farmer adoption. In a real agricultural system, a forecast has value only if it arrives in time to influence a decision and if the farmer understands how to act on it.

Therefore, the gap addressed by this thesis is not simply the absence of a forecasting algorithm. It is the absence of an end-to-end decision-support component that links multivariate forecasting, explainable recommendations, offline mobile access, and multi-channel alerts in the Sri Lankan fruit supply chain context. The contribution is software-engineering oriented: designing, implementing, and evaluating a usable forecasting service that can be integrated into the broader FreshRoute platform.

1.2.3 Research questions

The study is guided by one primary research question and several secondary questions. The primary question is: How can an integrated, AI-driven decision-support system reduce post-harvest waste and improve economic outcomes for Sri Lankan fruit farmers by mitigating information asymmetry?

Table 1. 2: Research questions and expected evidence

Research Question	Evidence Required
RQ1: To what extent can a multivariate LSTM model forecast short-term fruit demand in the Sri Lankan market?	Model performance measured using MAE, RMSE, MAPE, and WAPE on held-out test data.
RQ2: How does LSTM performance compare with SARIMA and Prophet baselines?	Comparative error metrics across the same fruits, districts, and forecast horizons.
RQ3: What architecture can collect, process, and serve forecasts in low-resource rural environments?	Backend architecture, database schema, API response time, cache behavior, and offline mobile validation.
RQ4: How can SMS and push notifications be designed for timely farmer alerts?	Alert delivery test cases, threshold rules, notification logs, and user feedback.
RQ5: What UI and UX principles support low-literacy and low-connectivity farmers?	Usability test observations, task completion rates, and qualitative feedback.
RQ6: What adoption barriers and facilitators influence use of FreshRoute?	User acceptance testing, stakeholder interviews, and pilot observations.
RQ7: What is the potential economic effect of forecast-guided harvest planning?	Estimated reduction in distress sales, improved price alignment, and reduced waste after pilot deployment.

1.3 Research Problem

Sri Lankan fruit farmers face a persistent inability to anticipate demand and price movement during the short period in which harvest decisions must be made. The biological perishability of fruits compresses the decision window. Once fruits reach harvest maturity, farmers cannot indefinitely delay sale without quality degradation. In the absence of predictive information, they may harvest at the same time as neighboring producers, creating oversupply and lowering prices. Alternatively, they may delay harvest without evidence and risk spoilage or quality decline.

This problem is aggravated by the structure of the supply chain. Intermediaries frequently control market access and possess better information about buyer demand. Farmers therefore negotiate from a weaker position. Moreover, current digital tools do not sufficiently transform historical data into future-oriented recommendations. The research problem can therefore be stated as follows: There is no accessible, integrated, AI-driven demand forecasting and alert system that enables Sri Lankan fruit farmers to align harvest and sales decisions with predicted market demand under rural connectivity and digital-literacy constraints.

Solving this problem requires addressing three interconnected technical and socio-technical challenges. The first is data integration: historical prices, weather, production cycles, and seasonal factors must be collected, cleaned, aligned by date and crop, and converted into features suitable for machine learning. The second is predictive modeling: the system must capture nonlinear temporal patterns while being compared against transparent baselines. The third is decision-support delivery: forecasts must be converted into concise advisories and delivered through mobile dashboards, SMS, and push notifications in a manner that farmers can understand and trust.

1.4 Research Objectives

1.4.1 Main objective

The main objective of this research is to design, develop, and evaluate an integrated real-time demand forecasting and alert system powered by artificial intelligence to help Sri Lankan fruit farmers make data-driven harvest planning and sales decisions. The system uses LSTM-based time-series modeling to analyze market, weather, and seasonal data and converts predictions into actionable advisories delivered through offline-capable mobile and SMS channels.

1.4.2 Specific objectives

1. To collect and prepare historical fruit market prices, demand indicators, weather observations, weather forecasts, and seasonal yield records from trusted sources such as economic centers, government datasets, agricultural reports, and weather APIs.
2. To design a robust data preprocessing pipeline that treats missing values, handles outliers, normalizes numerical variables, constructs lag features, encodes calendar effects, and prepares sequence windows for LSTM training.
3. To develop and train a multivariate LSTM model for short-term fruit demand and price forecasting, while comparing its performance against SARIMA and Prophet baselines using accepted forecasting metrics.
4. To implement a FastAPI-based prediction service that exposes secure REST endpoints for forecast retrieval, recommendation generation, and mobile application integration.
5. To design a PostgreSQL-backed data layer that stores users, crops, market observations, weather observations, forecast runs, forecast results, alert preferences, and notification logs in a normalized and maintainable schema.
6. To develop an offline-capable Android mobile dashboard using React Native to present forecasts, trends, confidence levels, explanations, and harvest recommendations in a farmer-friendly interface.
7. To implement SMS and Firebase push notification workflows that alert farmers about significant demand or price changes, with deep links to detailed forecast screens for smartphone users.
8. To evaluate the system through model validation, unit testing, integration testing, API performance testing, mobile usability testing, and user acceptance testing with representative farmer workflows.

1.4.3 Business and social objectives

In addition to the technical objectives, FreshRoute has business and social objectives. From a business perspective, it aims to support a scalable agricultural intelligence service that can later offer premium analytics, logistics coordination, and institutional dashboards. From a social perspective, it aims to improve farmer income stability, reduce avoidable fruit waste, and support a more transparent agricultural market. These objectives align with the broader goal of modernizing Sri Lanka's fruit supply chain without excluding rural farmers who have limited digital resources.

1.5 Scope of the Thesis

This thesis focuses primarily on the AI-driven demand forecasting and alert component of FreshRoute. The implementation scope includes data acquisition, preprocessing, LSTM model development, baseline comparison, forecast serving through APIs, PostgreSQL storage, mobile forecast display, offline caching, and SMS/push notification delivery. The broader FreshRoute platform includes marketplace, logistics, blockchain, and quality verification modules; these are discussed where they influence integration requirements, but their complete implementation is outside the individual scope of this thesis.

The initial crop scope includes high-priority fruit categories such as banana, mango, and papaya. The geographic scope is intended to support Sri Lankan market centers and fruit-producing districts, with Dambulla and other regional economic centers considered as important data and pilot contexts. The system is designed as a prototype suitable for final-year project validation and future pilot deployment, rather than as a fully commercial national-scale service at the time of writing.

1.6 Organization of the Thesis

Chapter 1 introduces the research background, literature, gap, problem, objectives, and scope. Chapter 2 presents the methodology, including system architecture, data methodology, model development, database design, mobile application design, commercialization aspects, implementation, testing, deployment, and feasibility. Chapter

3 presents the results and discussion, including model evaluation design, system testing results, research findings, limitations, and student contributions. Chapter 4 concludes the thesis and provides recommendations for future work. The reference list, glossary, and appendices provide supporting material such as sample API payloads, test cases, and data dictionaries.

2 METHODOLOGY

2.1 Methodology

2.1.1 Research and development approach

The methodology adopted for FreshRoute combines software engineering lifecycle practices with machine learning experimentation. A purely linear development process is unsuitable because requirements, data quality, and model behavior evolve as the system is tested. Therefore, the project follows an Agile-Scrum-inspired iterative approach supported by a structured machine learning workflow. Requirements are gathered and prioritized, prototype modules are developed in short cycles, outputs are reviewed, and improvements are incorporated into subsequent iterations.

The development process consists of seven broad stages: feasibility analysis, requirement gathering, system design, data preparation, implementation, testing, and deployment planning. This structure allows the project to remain academically systematic while retaining the flexibility required for AI-based development. For example, early data exploration may reveal missing values or inconsistent price formats, requiring changes to preprocessing logic before model training can proceed. Similarly, farmer feedback may require UI language or icon changes even after backend APIs are implemented.

The research methodology is also guided by the principle of socio-technical fit. The system is not judged only by model accuracy. It is judged by whether accurate predictions can be delivered securely, quickly, understandably, and reliably to farmers in real operating environments. Therefore, model evaluation, API validation, mobile testing, and user acceptance testing are treated as complementary forms of evidence.

2.1.2 Requirement gathering and stakeholder analysis

Requirements were derived from the project proposal, literature review, observed limitations of existing systems, and stakeholder needs in the fruit supply chain. The main stakeholder groups are farmers, buyers, transport providers, administrators, and institutional partners such as agricultural agencies. The forecasting component primarily

serves farmers but also provides value to buyers and administrators by improving the predictability of supply and reducing last-minute distress transactions.

The farmer requires simple and timely guidance: whether to harvest, delay, or prepare for high demand. The buyer requires visibility into expected supply and price direction. The administrator requires system monitoring, model performance tracking, alert audit logs, and the ability to update crop and market configurations. The technical requirements therefore include forecast generation, data integration, user-specific alert thresholds, offline mobile access, API security, and maintainable data storage.

Table 2. 1: Functional requirements of FreshRoute forecasting component

Requirement	Description	User Value
Demand forecast generation	Generate daily or weekly demand and price forecasts for selected fruits.	Supports harvest and sales planning.
Data integration	Collect market, weather, and seasonal yield data from trusted sources and APIs.	Improves forecast relevance and timeliness.
Forecast dashboard	Display trend charts, confidence levels, explanations, and recommendations.	Makes model output understandable to farmers.
Alerts and notifications	Send SMS and push notifications when demand or price thresholds are exceeded.	Allows rapid response without constant app checking.
Offline access	Cache latest forecast JSON and alerts locally on the device.	Supports rural use under intermittent connectivity.

User management	Allow users to register, log in, select crops, and set alert preferences.	Personalizes the system to farmer needs.
Analytics interface	Track forecast performance, notification delivery, and user engagement.	Enables continuous improvement and administration.

Table 2. 2: Non-functional requirements of the proposed system

Requirement	Expected Standard	Implementation Consideration
Usability	Interfaces should be understandable by users with limited digital literacy.	Use local language labels, icons, simple trend indicators, and clear action banners.
Performance	Forecast API should respond within an acceptable time for mobile use.	Use Redis caching, optimized queries, asynchronous jobs, and lightweight JSON responses.
Scalability	System should support additional crops, districts, and users.	Use modular services, normalized schema, containerized deployment, and scalable cloud resources.
Reliability	System should remain useful during data or network interruptions.	Cache latest successful data, handle API failures, and mark offline forecasts clearly.
Security	User data and API communication must be protected.	Use HTTPS over TLS, authentication, environment variables, input validation, and role-based authorization.

Maintainability	Code and models should be easy to update.	Use documentation, version control, modular packages, and reproducible training scripts.
-----------------	---	--

2.1.3 Overall system architecture

The FreshRoute demand forecasting architecture is organized into five major layers: data acquisition, preprocessing and feature engineering, model training and inference, application services, and user delivery channels. External data sources provide historical prices, weather observations, weather forecasts, and seasonal production indicators. These data are cleaned and transformed before being stored in PostgreSQL. The AI forecasting engine uses prepared sequence windows to train and run an LSTM model. The trained model is served through a FastAPI service that returns forecast JSON to mobile and web clients. The alert service monitors forecasts and triggers notifications through SMS and Firebase Cloud Messaging when thresholds are reached.

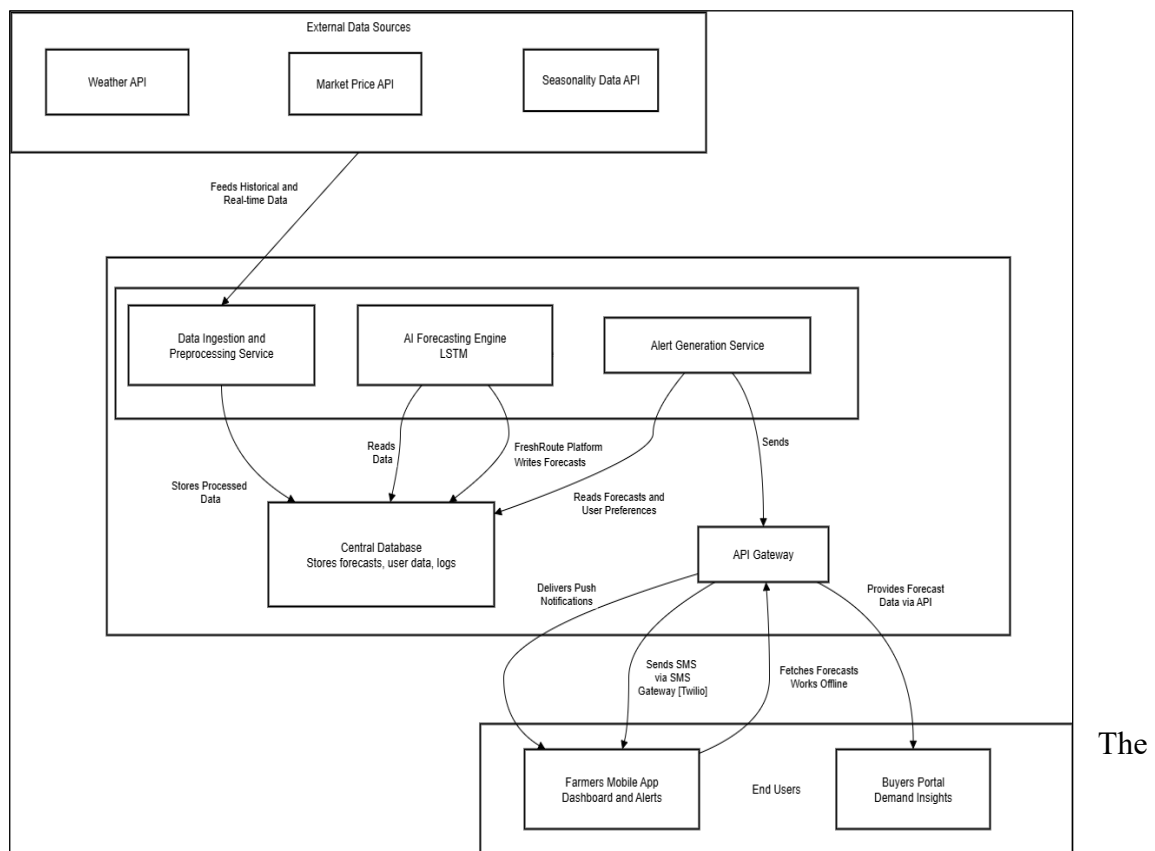


Figure 2. 1: High-level architecture of the FreshRoute demand forecasting system

system also supports integration with the broader FreshRoute platform. Forecast outputs can influence product listing recommendations, and marketplace events can later provide

additional training data. The architecture uses RESTful APIs and shared identifiers such as `user_id`, `crop_id`, `market_id`, `forecast_run_id`, and `listing_id` to maintain traceability across modules. This approach ensures that the forecasting module can function independently during early development while remaining compatible with future marketplace, logistics, and quality verification services.

2.1.4 Data collection methodology

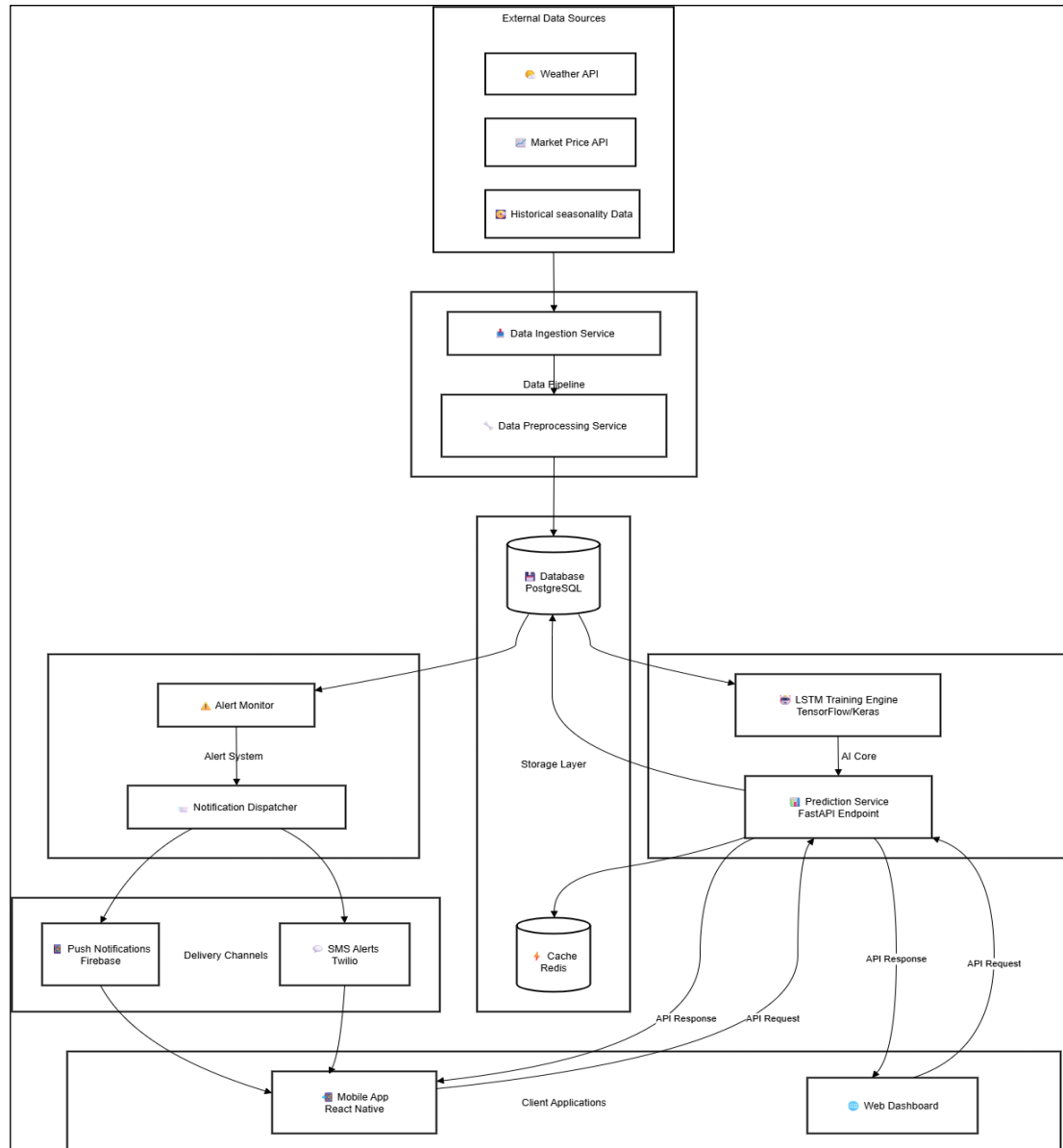


Figure 2. 2: System overview and data flow

The accuracy of the forecasting system depends strongly on the quality, granularity, and representativeness of its input data. The project uses multiple data sources to capture the main factors that influence fruit demand and price movement. Historical market prices reflect supply-demand interaction at economic centers. Weather observations and forecasts capture environmental factors that influence yield, harvest timing, transport conditions, and consumer demand. Seasonal yield records help the model distinguish regular production cycles from abnormal market shocks.

Table 2. 3: Data sources used by the forecasting module

Data Type	Potential Source	Examples of Variables	Collection Method
Market price data	Economic center portals, government agricultural reports, market board records	Crop, variety, market, date, wholesale price, retail price, volume indicator	Web scraping, CSV import, manual upload, or API integration
Weather observations	Department of Meteorology, OpenWeatherMap, or similar weather APIs	Temperature, rainfall, humidity, wind speed, weather condition	Scheduled API requests and historical dataset import
Weather forecasts	Weather APIs and meteorological services	Forecasted rainfall, temperature, humidity for next 5-7 days	API integration at regular intervals
Seasonal yield data	HARTI, Department of Census and Statistics, agriculture ministry reports	District, crop, season, estimated production, cultivated area	Report extraction, CSV entry, and administrative data import
System transaction data	FreshRoute application database after deployment	Listings, orders, accepted prices, alert	Automatically recorded through platform usage

		responses, delivery outcomes	
--	--	---------------------------------	--

The data collection process must also account for practical limitations. Public agricultural datasets may contain missing days, inconsistent crop names, different units of measurement, or district-level aggregation that does not directly map to a particular farmer's market. Therefore, the data pipeline includes standardization rules for crop naming, market identifiers, date formats, and units. A data dictionary is maintained in the appendices to ensure that each variable has a documented meaning and expected range.

2.1.5 Data preprocessing and feature engineering

Raw agricultural and weather data are rarely ready for direct model training. The preprocessing phase transforms heterogeneous records into aligned time-series sequences. First, all records are converted into a common date format and mapped to crop and market identifiers. Missing values are treated using interpolation, forward filling, or domain-specific rules depending on the variable. For example, a missing market price between two valid observations may be interpolated, while a missing rainfall value may be treated differently depending on available weather records.

Outlier detection is performed to reduce the influence of erroneous records. Market prices that deviate sharply from historical ranges are flagged and reviewed. Some outliers may represent real market shocks and should not be removed blindly; therefore, the pipeline distinguishes between likely data-entry errors and plausible extreme events. Numerical variables are scaled using Min-Max scaling or standardization before LSTM training, while categorical variables such as crop type, market, and season are encoded appropriately.

Feature engineering is central to the forecasting model. Lag features are created to represent previous prices and demand indicators over selected windows such as 7, 14, 30, and 60 days. Rolling averages and rolling volatility features capture trend and instability. Calendar features such as day of week, month, season, public holidays, and festival

periods help identify recurring demand cycles. Weather features include both current observations and forward-looking forecasts, since expected rainfall or heat may influence transport, supply, and consumer behavior.

Table 2. 4: Data preprocessing and feature engineering operations

Operation	Purpose	Expected Output
Date alignment	Ensure all records use a common temporal index.	Daily crop-market time series.
Missing value treatment	Reduce gaps that can break model training.	Complete or flagged sequences.
Outlier handling	Identify unrealistic price or weather values.	Cleaned dataset with retained legitimate shocks.
Normalization	Scale numerical features for neural network training.	Features in comparable numerical ranges.
Lag construction	Represent historical dependencies.	Past price, demand, and weather features over defined windows.
Rolling statistics	Capture trend and volatility.	Rolling mean, standard deviation, and momentum indicators.
Calendar encoding	Capture seasonality and recurring demand.	Month, week, season, holiday, and festival indicators.
Sequence windowing	Prepare data for LSTM input.	Three-dimensional tensors: samples, timesteps, features.

2.1.6 LSTM model development

The primary forecasting model is a multivariate LSTM network. The model receives a sequence of historical observations for a crop-market combination and predicts a short-term target such as next-day price, next-week demand index, or expected demand level. The input tensor has the structure samples x timesteps x features. The timesteps dimension may represent the most recent 30 or 60 days, while the feature dimension contains normalized price, weather, calendar, and seasonal variables.

The basic architecture consists of one or two LSTM layers with 50-100 hidden units, followed by dropout layers to reduce overfitting and a dense output layer with linear activation for continuous prediction. The model is trained using the Adam optimizer and Mean Squared Error loss. Early stopping is used to halt training when validation loss stops improving. This prevents the model from memorizing training data and improves generalization to unseen periods.

Model development follows a reproducible workflow. The dataset is divided into training, validation, and test sets using chronological splitting rather than random shuffling, because time-series data must preserve temporal order. Hyperparameters such as sequence length, number of LSTM units, dropout rate, batch size, learning rate, and forecast horizon are tuned through validation experiments. The final model and fitted scaler are saved as versioned artifacts, such as `model.keras` and `scaler.pkl`, so that inference uses the same transformations as training.

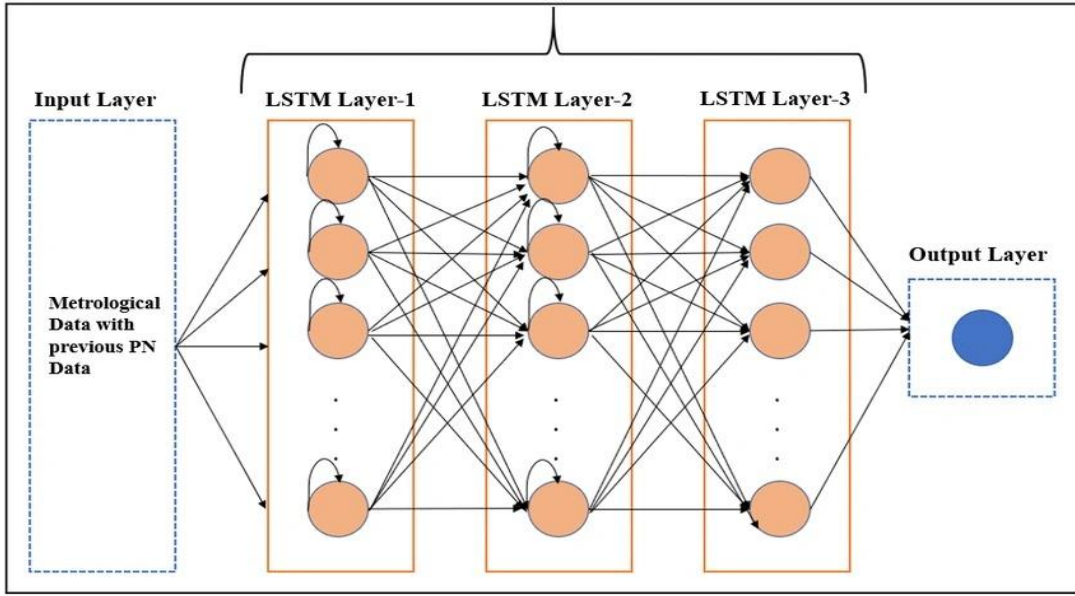


Figure 2. 3: LSTM model development structure

2.1.7 Baseline models and evaluation metrics

The LSTM model is compared with SARIMA and Prophet baselines. SARIMA provides a statistical benchmark for seasonal time-series behavior. Prophet provides an interpretable decomposition-based benchmark suitable for trend and seasonality. Comparing against these baselines is necessary because a complex neural model should be adopted only if it provides practical improvement over simpler alternatives.

Forecast performance is evaluated using Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Mean Absolute Percentage Error (MAPE), and Weighted Absolute Percentage Error (WAPE). MAE provides an easily interpretable average error in the original unit. RMSE penalizes large errors more heavily and is useful when large forecasting mistakes are especially costly. MAPE expresses error as a percentage but can be unstable when actual values are near zero. WAPE is useful for aggregated demand contexts because it weights absolute error by actual volume.

In addition to numerical accuracy, the system evaluates advisory usefulness. A forecast is valuable only if it changes decisions. Therefore, pilot evaluation should measure whether farmers viewed forecasts, whether alerts were opened, whether harvest timing changed,

and whether produce was sold under better conditions. These behavioral metrics are included in the results framework.

2.1.8 Backend and API design

The backend is designed around FastAPI, a Python framework suitable for building high-performance REST APIs. FastAPI supports request validation using typed schemas, automatic OpenAPI documentation, asynchronous endpoints, and efficient JSON serialization. These features are useful for a research prototype that must remain readable and maintainable while supporting mobile and web clients.

The prediction service exposes endpoints such as POST /predict, GET /forecast/{crop_id}, GET /forecast/history, POST /alert-preferences, and GET /notifications. A prediction request may include crop, market, district, forecast horizon, and recent observations. The service retrieves the required historical sequence from PostgreSQL, applies the saved scaler, runs the LSTM model, inverse-transforms the prediction, calculates trend and confidence indicators, stores the forecast result, and returns a structured JSON response.

The API response is intentionally designed for mobile consumption. It includes the predicted value, trend indicator, confidence level, forecast horizon, explanation text, and recommendation code. This prevents the mobile application from needing to interpret raw model output. It also allows the recommendation logic to be versioned and audited centrally on the backend.

2.1.9 PostgreSQL database design

PostgreSQL is selected as the primary application database because the system requires reliable relational storage, structured queries, indexing, and transactional consistency. The forecasting component stores both user-facing data and internal model records. Core tables include users, farmers, crops, markets, market_prices, weather_observations, weather_forecasts, seasonal_yields, forecast_runs, forecast_results, alert_preferences, and notification_logs.

The `market_prices` table stores crop, market, date, price, and source metadata. The `weather_observations` table stores location, date, temperature, rainfall, humidity, and source. The `forecast_runs` table records the model version, training data range, prediction time, and metrics. The `forecast_results` table stores the actual outputs served to users. This separation between forecast run and forecast result supports traceability: if a prediction is questioned later, administrators can identify which model version and input period produced it.

Indexes are required on frequently queried fields such as `crop_id`, `market_id`, `date`, `user_id`, and `forecast_run_id`. For time-series retrieval, composite indexes on `crop_id`, `market_id`, and `date` improve performance. PostgreSQL constraints protect data integrity by preventing duplicate records for the same crop-market-date combination. Foreign keys maintain relationships among users, crops, forecasts, and notifications.

2.1.10 Redis caching and performance considerations

Redis is used as an optional caching layer for recently requested forecasts. Mobile clients may frequently request the same crop and market forecast within a short period. Serving every request directly from the model would be inefficient and may increase latency. Therefore, forecast results are stored in PostgreSQL for persistence and cached in Redis for fast retrieval. Cache keys can be structured using crop, market, horizon, and model version. When a new forecast is generated or a model version changes, the corresponding cache entries are invalidated.

Performance design also includes background jobs for scheduled data collection, model retraining, and alert evaluation. The system should not train models synchronously in response to a mobile request. Instead, training and batch forecasting are executed as scheduled jobs, while the API serves the latest approved forecast. This design improves reliability and ensures that farmers receive fast responses even when the model training pipeline is computationally expensive.

2.1.11 Mobile application design and offline capability

The mobile application is developed using React Native to support Android deployment and future cross-platform expansion. The main farmer-facing screens are Login, Dashboard, Forecast Details, Alerts, Crop Preferences, and Settings. The Dashboard summarizes selected crops using simple cards that show predicted price direction, demand level, confidence, and recommendation. The Forecast Details screen displays trend charts, explanations, weather influence, and a clear action banner.

Offline capability is implemented by storing the latest forecast response, user preferences, and recent alerts on the device. When the application detects no internet connection, it displays cached forecasts with a visible timestamp and a warning that the data may not be live. When connectivity returns, the app synchronizes with the backend and refreshes the cache. This design is important for rural users because the system should remain useful even when real-time connectivity fails.

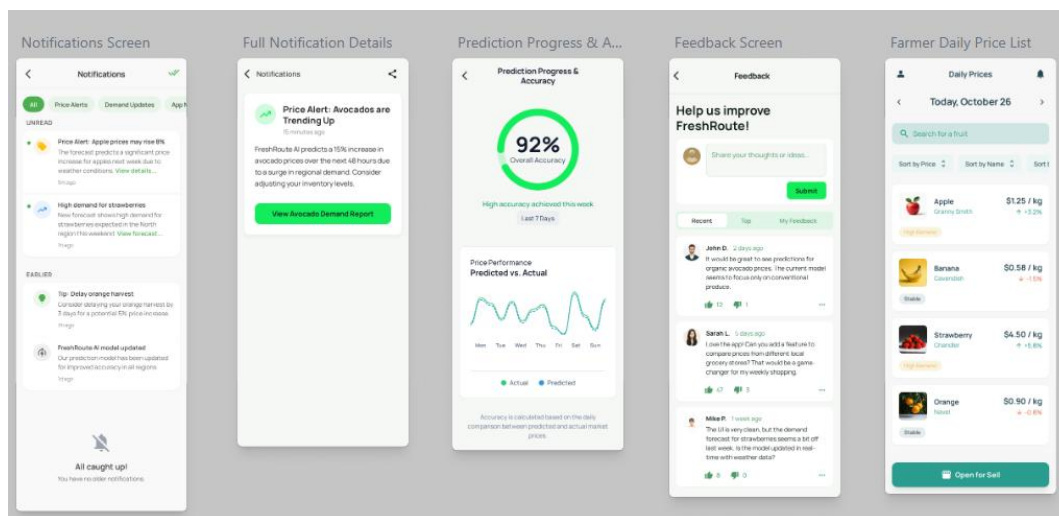


Figure 2. 4: Mobile forecast design UI

2.1.12 Alert generation workflow

The alert service continuously evaluates new forecast results against user-defined thresholds and predefined agronomic rules. For example, if a farmer has selected banana alerts and the forecast indicates a price increase greater than 10%, the system may send a

SELL_SOON or PREPARE_STOCK alert depending on the direction and context. If forecasted demand is expected to fall, the system may advise delaying harvest where biologically feasible.

Firebase Cloud Messaging is used for push notifications to smartphone users. Each push notification includes a title, short message, crop identifier, forecast identifier, and deep link such as `freshroute://forecast/banana`. When the user taps the notification, the app opens the relevant forecast screen. SMS alerts are sent through Twilio or an approved local gateway to farmers who have feature phones or prefer SMS. SMS content is intentionally concise because messages must be readable and cost-effective.

2.1.13 Security and privacy design

Security is a non-functional requirement because the platform stores user information, contact numbers, crop preferences, and potentially commercially sensitive market data. All communication between mobile clients, web clients, and backend APIs should use HTTPS protected by Transport Layer Security (TLS). TLS encrypts data in transit, preventing attackers from reading credentials, API responses, or alert preferences over the network. SSL is often used informally to refer to secure web communication, but modern systems should use TLS 1.2 or TLS 1.3 rather than obsolete SSL protocols.

Authentication is implemented using secure login credentials and token-based sessions. API keys for weather services, SMS providers, Firebase, and databases are stored as environment variables rather than being hard-coded in source code. Role-based access control separates farmer, buyer, administrator, and system-service permissions. Input validation protects API endpoints from malformed requests, and audit logs record sensitive administrative actions such as model approval, alert configuration changes, and data imports.

Privacy design follows data minimization. The model does not require unnecessary personal information to generate forecasts. Personally identifiable information should be separated from analytical datasets wherever possible. Aggregated and anonymized data

can be used for model improvement and policy insights, while individual user contact details remain protected.

2.1.14 Integration with broader FreshRoute modules

The integrated FreshRoute research paper describes additional modules that complement forecasting. The permissioned marketplace uses Hyperledger Fabric to record high-value state transitions such as escrow commitments and delivery confirmations. Hyperledger Fabric is appropriate for known supply chain participants because it supports permissioned membership, certificate-based identity, modular consensus, and private channels [14]. In the FreshRoute context, the forecasting module does not need to store raw predictions on-chain. Instead, it can provide advisory context to marketplace listings, while critical trade commitments are recorded by the marketplace service.

The logistics module uses perishability-aware routing rules, considering travel distance, temperature sensitivity, ethylene gas compatibility, and delivery timing. Forecasting can support logistics by predicting supply concentration and enabling earlier vehicle planning. The quality module uses image-based verification and complaint evidence. Forecasting can also benefit from downstream quality and delivery outcomes, because repeated spoilage or failed deliveries may indicate that certain forecast-guided decisions require adjustment.

This integration is implemented through shared identifiers and event-driven APIs. A produce lot created in the marketplace can reference a `forecast_result_id` that influenced the listing. A notification log can reference whether an alert led to a listing or action. Over time, these connections allow FreshRoute to evaluate whether forecast advisories actually reduce waste and improve farmer outcomes.

2.1.15 Tools and technologies

Table 2. 5: Tools and technologies used in implementation

Area	Tool/Technology	Implementation Context
Version control	Git and GitHub	Source code management, branch-based collaboration, issue tracking, and release tagging.
Project management	Jira	Sprint planning, task assignment, backlog tracking, and milestone monitoring.
UI/UX design	Figma	Wireframes, mobile screen prototypes, and usability review artifacts.
Model development	Python, TensorFlow, Keras, Scikit-learn, Pandas, NumPy	Data preprocessing, LSTM training, baseline evaluation, and model serialization.
Development environment	Google Colab, VS Code, PyCharm	GPU-supported experimentation and software development.
Backend	FastAPI	REST API endpoints for forecasting, alert preferences, users, and notifications.
Database	PostgreSQL	Relational storage for users, crops, prices, weather, forecasts, and notification logs.
Caching	Redis	Fast retrieval of latest forecasts and reduction of

		repeated database/model calls.
Mobile application	React Native	Android mobile dashboard with offline caching and alert deep links.
Notifications	Firebase Cloud Messaging and Twilio or local SMS gateway	Push notifications for smartphones and SMS alerts for feature phones.
Deployment	Docker and cloud VM such as AWS EC2, GCP, or Azure VM	Containerized backend and model services with environment-based configuration.
Security	HTTPS/TLS, JWT or token-based authentication	Encrypted communication and controlled API access.
Testing	Pytest, Jest, Postman, optional Playwright for web portal E2E tests	Unit, integration, API, mobile, and browser workflow validation.

2.2 Commercialization Aspect of the Product

2.2.1 Value proposition

The commercialization potential of FreshRoute is based on a clear value proposition: farmers receive timely, localized, and actionable forecasting intelligence that can reduce distress selling and avoidable waste, while buyers and supply chain actors benefit from improved visibility into upcoming supply. The system does not attempt to replace farmers' judgment. Instead, it provides evidence that supports better timing, negotiation, and planning.

For smallholder farmers, the free or low-cost version of FreshRoute can provide short-term forecasts, basic trend indicators, and important alerts. For medium and large-scale

producers, cooperatives, exporters, and agribusinesses, premium analytics can provide longer forecast horizons, multi-crop dashboards, regional comparisons, and planning reports. For institutions, aggregated data can support market monitoring, food security planning, and post-harvest intervention targeting.

2.2.2 Business model

The proposed business model follows a freemium strategy. Core features are offered free or subsidized to encourage farmer adoption. Revenue can be generated through FreshRoute Pro subscriptions, modest transaction fees in the broader marketplace, logistics coordination charges, institutional dashboards, and data analytics services using anonymized aggregated data. Care must be taken to ensure that monetization does not recreate the same information inequality that the platform is designed to reduce.

Table 2. 6: Commercialization strategy summary

Element	Strategy	Rationale
Free farmer access	Offer basic forecasts, simple advisories, and limited alerts at no cost.	Encourages adoption among smallholders and creates network effects.
Premium analytics	Provide long-range forecasts, multi-crop comparisons, and advanced alerts.	Generates revenue from users who gain higher commercial value.
Telecom partnerships	Bundle SMS and data with Dialog, Mobitel, or other providers.	Reduces alert costs and improves rural reach.
Government and institutional partnerships	Align with Ministry of Agriculture, HARTI, NIPHM, cooperatives, and economic centers.	Improves trust, access to data, and field deployment support.

Agribusiness integration	Provide APIs and dashboards for buyers, exporters, and logistics partners.	Creates enterprise value while strengthening farmer-market linkages.
Anonymized analytics	Offer aggregated insights to policy and planning stakeholders.	Supports sustainability without exposing personal farmer data.

2.2.3 Market entry and rollout strategy

The initial rollout should focus on fruit-growing clusters where supply is frequent and market volatility is visible. Banana, mango, and papaya are suitable early crops because they are economically significant and commonly affected by timing, perishability, and market fluctuations. The first deployment should involve a limited farmer group, one or two market centers, and a small set of buyers or extension officers who can validate recommendations.

Training and trust building are essential. Farmers may be skeptical of AI-based advice, especially if predictions conflict with informal market expectations. Therefore, demonstrations should explain forecasts using simple terms, show historical examples, and emphasize that FreshRoute provides guidance rather than absolute certainty. Local language support and field-agent assistance can increase adoption.

2.2.4 Sustainability and social impact

Long-term sustainability depends on maintaining the cost of coordination below the value gained from reduced waste, better prices, and improved market access. SMS costs, cloud hosting, model retraining, and support services must be carefully managed. Institutional pilots can subsidize early adoption until there is enough transactional density for premium services or marketplace fees to support operations.

The expected social impact includes improved farmer decision-making, reduced dependence on opaque intermediaries, better use of market data, and reduced post-harvest losses. If scaled responsibly, the platform can contribute to food security and income

stability. However, data governance must remain transparent to avoid creating new digital intermediaries that farmers do not understand or control.

2.3 Testing and Implementation

2.3.1 Implementation phases

The implementation is divided into five phases. Phase 1 focuses on data infrastructure and model experimentation. PostgreSQL tables are created, data ingestion scripts are developed, preprocessing functions are implemented, and baseline models are trained. Phase 2 focuses on backend development, including FastAPI endpoints, prediction service integration, alert preference management, and notification logging. Phase 3 focuses on mobile application development, including authentication, dashboard screens, forecast details, settings, and offline caching. Phase 4 integrates all components and conducts end-to-end testing. Phase 5 prepares deployment through Docker, cloud hosting, monitoring, and user acceptance testing.

2.3.2 Model training implementation

The model training pipeline is implemented in Python. The preprocessing script loads raw market, weather, and seasonal data; applies cleaning rules; creates features; scales numerical variables; and generates sequence windows. The training script builds the LSTM architecture, trains the model on the training set, monitors validation loss, and saves the best model. Baseline scripts train SARIMA and Prophet models for the same forecast horizon. Evaluation scripts calculate MAE, RMSE, MAPE, and WAPE on the test set and export comparison tables for the results chapter.

2.3.3 API implementation

The FastAPI application separates routes, services, schemas, and database repositories. Pydantic schemas validate request and response formats. The prediction service loads the trained model and scaler at startup or through a model registry path. Forecast endpoints retrieve recent historical data, run inference where required, or return cached forecast

results. The alert endpoint allows users to configure crop-specific thresholds and notification preferences.

API security is implemented through authentication tokens and HTTPS deployment. In local development, environment variables are loaded from a secure configuration file that is not committed to the repository. In deployment, secrets are stored in the cloud environment or container runtime. API documentation generated by FastAPI helps test the service and communicate endpoint behavior to team members.

2.3.4 Mobile implementation

The React Native mobile application consumes REST APIs and stores selected data locally. The dashboard prioritizes visual clarity. Instead of showing only numerical predictions, it displays a trend arrow, confidence label, recommendation banner, and short explanation. For example, a recommendation may state: "Delay harvest by two days because the forecast indicates a temporary price decrease and rainfall is expected." The exact wording should be localized into Sinhala and Tamil where possible.

Offline synchronization is implemented using local storage. When the app opens, it checks connectivity. If connected, it fetches the latest forecasts and stores them locally. If offline, it loads the most recent cached data and displays an offline indicator. This ensures that the application remains useful even when internet access is unavailable.

2.3.5 Testing strategy

The testing strategy covers model, backend, mobile, integration, performance, security, and user acceptance concerns. Unit tests verify small functions such as missing value treatment, normalization, feature generation, API request validation, and recommendation mapping. Integration tests verify that the mobile app, backend API, database, model service, and notification service work together. Performance tests evaluate response time under concurrent requests. User acceptance testing evaluates whether farmers can interpret the dashboard and act on alerts.

Table 2. 7: Test plan for the forecasting and alert system

Test Area	Objective	Expected Outcome
Data preprocessing	Verify that missing values, outliers, normalization, and encoding are handled correctly.	Clean dataset ready for training with documented transformations.
LSTM accuracy	Validate predictive accuracy using held-out historical data.	Error metrics meet the acceptable threshold defined for pilot deployment.
Baseline comparison	Compare LSTM against SARIMA and Prophet.	Evidence of whether LSTM adds practical value over baselines.
Real-time forecast API	Submit a forecast request using latest sequence data.	API returns valid JSON with prediction, trend, confidence, and recommendation.
Alert triggering	Test user threshold rules for price or demand changes.	SMS and push notifications are created and logged correctly.
Mobile usability	Test login, dashboard, forecast details, settings, and offline behavior.	Users complete core tasks and cached data displays correctly offline.
Integration	Trace forecast request from mobile app to backend and database.	Data remains consistent across all system components.
Load testing	Simulate 100 or more concurrent API requests.	System maintains stable response time and resource usage.

Security testing	Validate TLS deployment, authentication, and protected endpoints.	Unauthorized access is blocked and sensitive data is not exposed.
------------------	---	---

2.3.6 Playwright and end-to-end web testing context

Although the main farmer interface is mobile, the broader FreshRoute ecosystem may include buyer and administrator web portals. For these browser-based workflows, Playwright can be used as an end-to-end testing framework. Playwright automates real browsers and can validate user journeys such as administrator login, forecast result review, crop configuration, user management, and notification log inspection. This type of testing is valuable because it confirms that frontend behavior, backend API responses, authentication, and database changes work together from the perspective of a real user.

Playwright tests should be executed in a staging environment with test accounts and seeded data. The tests can verify that protected pages require login, forecast tables load within an acceptable time, filters work correctly, and form submissions produce expected API calls. Screenshots and traces generated by Playwright can help identify UI regressions before deployment. This complements Pytest and Postman because it validates full browser workflows rather than isolated backend functions.

2.3.7 Deployment plan

The backend and model services are containerized using Docker. Containerization ensures that the same Python version, dependencies, and runtime configuration are used in development, testing, and deployment. The application can be deployed on a cloud virtual machine such as AWS EC2, Google Cloud Platform, or Azure VM. PostgreSQL can run as a managed database service or as a secured container during prototype deployment. Redis can be deployed as a local container or managed cache depending on budget and scale.

Deployment should use HTTPS with TLS certificates, environment-based secrets, and firewall rules that expose only required ports. Monitoring should collect API errors,

response time, notification failures, and model service exceptions. A scheduled retraining process can be configured after sufficient new data are collected. The mobile application can be distributed internally for pilot testing before publication on the Google Play Store.

2.3.8 Feasibility and risk management

The technical feasibility of FreshRoute is supported by the availability of mature open-source technologies. TensorFlow, Keras, FastAPI, PostgreSQL, React Native, Firebase, and Docker are widely used and well documented. The main technical risk lies not in tool availability but in data quality and model generalization. Agricultural datasets may be incomplete, inconsistent, or too sparse for certain fruits. This risk is mitigated through careful preprocessing, baseline models, rolling validation, and incremental deployment.

Economic feasibility is supported by the use of open-source software and student-accessible cloud resources. Costs arise from SMS credits, cloud hosting, domain or deployment resources, and field testing. Operational feasibility depends on user adoption. The app's offline mode, SMS fallback, and simple recommendation design reduce barriers for rural users. Schedule feasibility is achieved through phased implementation aligned with the academic timeline.

Key risks include poor data availability, API provider changes, SMS delivery failures, cloud cost overruns, farmer distrust, and model error during abnormal market shocks. Mitigation strategies include data validation, fallback models, alert frequency limits, transparent recommendation explanations, field training, and manual review of model performance before large-scale deployment.

3 RESULTS AND DISCUSSION

3.1 Results

3.1.1 Overview of evaluation

The evaluation of FreshRoute is designed to assess both technical correctness and practical usefulness. Because the proposal-stage documents do not provide final numeric field results, this thesis draft includes result structures and placeholders that should be completed after final model testing and user acceptance testing. The evaluation framework includes model accuracy, baseline comparison, API performance, alert delivery, offline mobile functionality, and user feedback. This approach follows the broader FreshRoute research paper, which recommends module-level and integrated evaluation rather than relying solely on laboratory metrics.

The component-level evaluation focuses on the demand forecasting module. The system-level evaluation examines how forecasts support farmer decisions and integrate with marketplace, logistics, and quality workflows. The two levels are complementary. A model can show good accuracy but fail in practice if alerts are delayed or the UI is difficult to understand. Conversely, a simple model can be useful if it delivers timely and understandable guidance. Therefore, the results are organized around both technical metrics and decision-support outcomes.

3.1.2 Data preparation results

The data preparation stage produced a structured dataset suitable for time-series modeling. Market price records, weather observations, weather forecasts, and seasonal indicators were aligned by crop, market, and date. Missing values and outliers were identified, and preprocessing transformations were applied to create normalized model inputs. The final dataset should be summarized in the table below after final data collection is completed.

Table 3. 1: Evaluation metrics and interpretation

Metric	Formula/Meaning	Interpretation
MAE	Average absolute difference between predicted and actual values.	Lower values indicate smaller average forecasting error.
RMSE	Square root of average squared error.	Penalizes large errors and highlights major forecasting failures.
MAPE	Average percentage error relative to actual values.	Useful for interpretability but unstable when actual values are near zero.
WAPE	Total absolute error divided by total actual value.	Useful for aggregate demand and volume-sensitive evaluation.
API response time	Time between request and response.	Indicates whether forecasts can be served efficiently to mobile users.
Alert delivery success rate	Delivered notifications divided by attempted notifications.	Measures reliability of SMS and push channels.
Task completion rate	Users completing a scenario divided by total test users.	Measures usability of dashboard and alert workflows.

3.1.3 Forecasting model results

The LSTM model is expected to be compared with SARIMA and Prophet using the same test periods and forecast horizons. The final report should insert numerical values after running the evaluation scripts. The anticipated interpretation should not be based only on which model has the lowest error. The selected model must also be stable, maintainable, and suitable for serving in a production-style API. If LSTM performs best for fruits with rich data but Prophet performs better for sparse datasets, the system may adopt a hybrid strategy in which the best model is selected per crop and market.

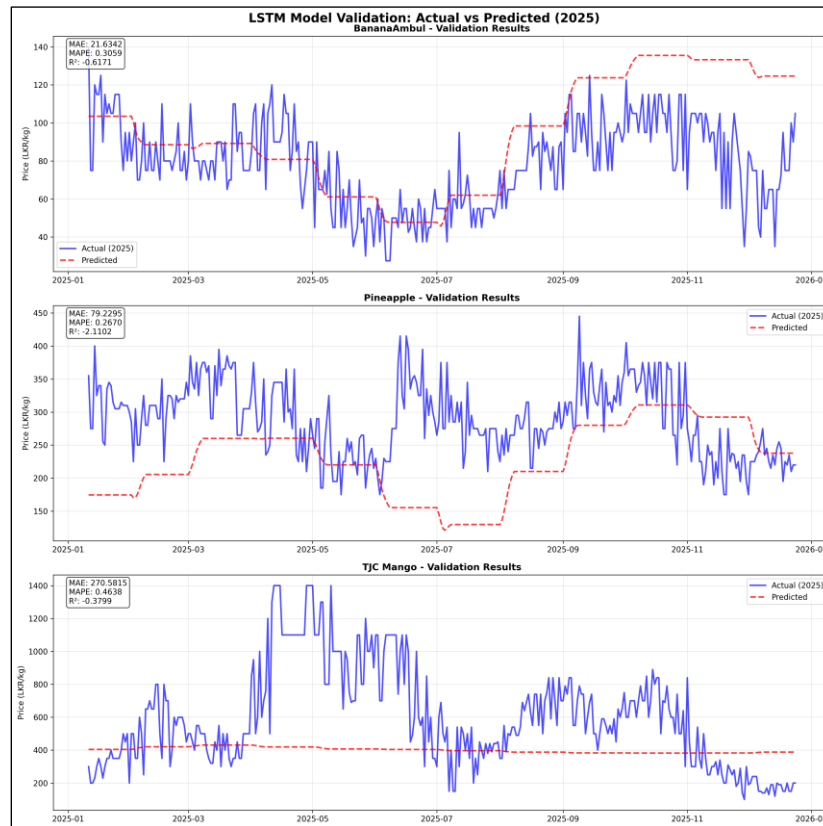


Figure 3. 1: Forecast Model Validation

The expected result narrative should discuss whether the LSTM model captured seasonal and nonlinear movements more effectively than baselines. It should also discuss failure cases. For example, sudden supply shocks, transport disruptions, or policy changes may

not be predictable from historical data. In such cases, explanation text and confidence indicators are important so that farmers understand the forecast as guidance rather than certainty.

3.1.4 Backend and API results

The backend evaluation measures whether the system can serve forecast results reliably and quickly. Test cases should verify successful request validation, correct database retrieval, correct use of model artifacts, response serialization, and error handling. Performance testing should simulate concurrent users to confirm that the API remains stable. The target proposal is that common API responses should be returned within approximately two seconds under a moderate prototype load of 100 or more concurrent requests. Final measured values should be inserted after load testing.

3.1.5 Alert and notification results

The alert workflow is evaluated by configuring threshold rules and generating forecasts that cross those thresholds. For example, if a farmer sets a banana alert threshold of 10% and the forecast indicates a 15% price increase, the alert service should create a notification record and dispatch both SMS and push messages if the user has enabled both channels. The notification log should record user ID, crop ID, forecast ID, channel, status, timestamp, and error message if delivery fails.

The evaluation should measure delivery success rate, delivery latency, duplicate prevention, and message clarity. SMS messages must be concise and should avoid technical terms. Push notifications should include deep links to the relevant forecast screen. In offline conditions, the mobile app should display received alerts after synchronization and should not lose previously cached forecast data.

3.1.6 Mobile usability results

Mobile usability testing should involve representative users performing common tasks such as logging in, selecting a crop, viewing a forecast, interpreting a recommendation, changing alert settings, and reading cached data offline. The key result is whether farmers can understand the recommendation without requiring technical interpretation of graphs. The final thesis should insert task completion rates, user feedback summaries, and screenshots of the final mobile screens.



Figure 3. 4: User acceptance feedback Collection

3.1.7 Integrated platform results

The broader FreshRoute research paper reports that the integrated architecture is evaluated across forecasting, marketplace matching, logistics optimization, and quality verification. The forecasting module is evaluated using WAPE, RMSE, MAE, forecast accuracy, and prediction time. The marketplace is evaluated using match acceptance rate, stock utilization rate, time-to-settlement, and escrow default rate. The logistics module is evaluated using route time, delivery delay, temperature violation flags, and freshness cost.

The quality module is evaluated using precision, recall, F1-score, detection accuracy, and quality agreement rate.

For the forecasting component, the integrated value lies in the fact that predictions can influence listing behavior and downstream coordination. Final field evaluation should therefore compare manual coordination against FreshRoute-supported workflows. Useful indicators include reduction in last-minute distress sales, improvement in price alignment, reduction in unsold stock, and user-reported confidence in harvest timing.

3.2 Research Findings

The findings below summarize the expected interpretation of the research questions based on the system design and evaluation framework. Final quantitative values should be inserted after experiments and pilot testing are completed.

3.3 Discussion

3.3.1 Technical interpretation

The proposed solution demonstrates how software engineering can translate machine learning into an operational decision-support workflow. The LSTM model is only one part of the system. Data pipelines, APIs, database schema, caching, notification logic, mobile design, security, and deployment practices are equally important. Without these supporting layers, a trained model would remain a laboratory artifact and would not reach the farmer at the moment of decision.

The choice of PostgreSQL supports data integrity and traceability. Agricultural forecasting data must be auditable because recommendations may influence financial decisions. By storing forecast runs, model versions, and notification logs, the system can later investigate why a particular advice was given. Redis improves performance but does not replace persistent storage. FastAPI provides an efficient mechanism for serving predictions and documenting API behavior. React Native supports a mobile-first approach suitable for farmer-facing deployment.

3.3.2 Practical interpretation for farmers

The major practical contribution is the conversion of predictive analytics into understandable advisories. Farmers do not need to know the details of LSTM gates, loss functions, or normalization. They need to know whether to harvest now, delay, or monitor conditions. Therefore, the interface intentionally presents recommendations before technical detail. Charts and explanations remain available for users who want more information, but the first screen emphasizes action.

SMS delivery is especially important for inclusiveness. A solution that works only through a data-intensive smartphone application would risk excluding the farmers most affected by information asymmetry. SMS alerts may not carry rich graphs, but they can deliver the essential message: crop, market, direction, and action. This makes the system more appropriate for rural deployment.

3.3.3 Discussion of security and trust

Trust is both a technical and social issue. Technical trust is supported through HTTPS/TLS, authentication, secure handling of API keys, database constraints, and audit logs. Social trust is supported through transparent recommendations, explanations, pilot training, and the ability to compare forecasts with actual outcomes. Farmers are more likely to adopt the system if they can see that recommendations are based on data and if errors are acknowledged rather than hidden.

The broader FreshRoute platform may use Hyperledger Fabric for auditable marketplace events. However, the forecasting module does not need to place large model features or raw datasets on a blockchain. A hybrid architecture is more appropriate: high-value transaction events can be recorded on a permissioned ledger, while high-volume data such as weather records, model features, and notification logs remain in PostgreSQL or object storage. This preserves performance while maintaining accountability.

3.3.4 Limitations

The main limitation is data availability. Forecasting accuracy depends on clean, consistent, and sufficiently long time series. Some fruit categories or markets may lack enough data for a reliable LSTM model. Missing values, informal transactions, and inconsistent units can introduce bias. Another limitation is that historical data cannot fully predict sudden shocks such as extreme weather, disease outbreaks, road closures, fuel shortages, or policy changes.

A second limitation is causal attribution. If farmers using FreshRoute achieve better prices, it does not automatically prove that the forecast caused the improvement. Seasonal changes, buyer behavior, training effects, and external market conditions may also contribute. Stronger evaluation requires baseline periods, comparison groups, and field-level outcome measurement.

A third limitation is adoption. Farmers may not immediately trust AI-generated recommendations, particularly if they have long relied on intermediaries. Training, local language support, explanation text, and gradual rollout are necessary. The system should avoid presenting predictions as absolute truth. Confidence indicators and clear disclaimers help users treat advisories as decision support.

3.3.5 Future enhancement

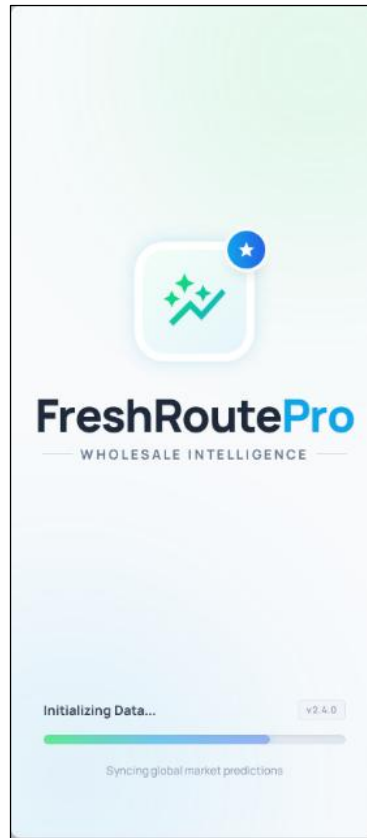


Figure 3. 5: Future Works with pro Version

Future work can extend the model to more fruits, more districts, and longer forecast horizons. Additional data such as fuel prices, holiday demand, import/export indicators, and real FreshRoute transaction data can improve prediction quality. Model explainability can be improved through feature importance analysis and natural language explanations. The mobile app can be extended with Sinhala and Tamil voice prompts for farmers with reading difficulties.

The broader platform can integrate forecast-guided listings, automatic buyer matching, logistics route planning, and quality verification. As transaction data accumulate, the system can learn which recommendations produce better outcomes and recalibrate models accordingly. Long-term research should estimate environmental impact, including reduced food waste and potential carbon savings from better logistics coordination.

4 CONCLUSION

4.1 Conclusion

This thesis presented the design, development, and evaluation framework for FreshRoute, an AI-driven demand forecasting and alert system for Sri Lanka's fruit supply chain. The research addressed a practical and economically significant problem: smallholder farmers often make harvest and sales decisions without reliable visibility into future demand and price movement. This information asymmetry contributes to oversupply, distress selling, post-harvest losses, and dependence on intermediaries.

The proposed system responds to the problem through a multivariate LSTM forecasting model supported by a complete software architecture. Historical market prices, weather data, and seasonal indicators are collected and transformed into model-ready sequences. The trained model generates short-term forecasts, which are then converted into farmer-facing recommendations. These recommendations are delivered through an offline-capable React Native mobile application, SMS alerts, and Firebase push notifications. The backend uses FastAPI for REST services, PostgreSQL for structured storage, Redis for caching, Docker for deployment consistency, and HTTPS/TLS for secure communication.

The main contribution of the thesis is not limited to the LSTM model. It is the end-to-end integration of data engineering, forecasting, API design, mobile usability, alert delivery, and deployment planning into a decision-support component suitable for rural agricultural contexts. By emphasizing offline access and SMS fallback, the system recognizes that effective digital agriculture must be inclusive and practical rather than only technologically advanced.

The evaluation framework defines how the system should be validated through forecasting metrics, baseline comparison, API performance testing, notification delivery testing, mobile usability evaluation, and user acceptance testing. Final numeric results should be inserted after completing the final experimental run and pilot validation. Nevertheless, the

methodology provides a clear path for demonstrating whether FreshRoute improves forecast accuracy, decision confidence, and supply chain coordination.

4.2 Recommendations

Several recommendations arise from this research. First, the forecasting model should be deployed gradually and compared continuously against baselines. LSTM should be used where it demonstrably improves performance, while simpler models should remain available for sparse data conditions. Second, data governance must be treated as a core design concern. Model versions, data sources, forecast results, and alerts should be auditable so that users and administrators can understand system behavior.

Third, farmer training should be included in any pilot deployment. Users must understand that forecasts are probabilistic guidance, not guaranteed outcomes. Fourth, local language support should be strengthened before large-scale use. Fifth, partnerships with agricultural institutions, economic centers, and telecommunications providers should be pursued to improve data access, trust, and SMS affordability.

4.3 Future Work

Future work should focus on field deployment and empirical impact measurement. A controlled pilot involving selected farmers, buyers, and market centers can compare FreshRoute-supported workflows against existing manual practices. Data should be collected on harvest timing, sale price, unsold stock, alert response, and farmer satisfaction. This will allow more confident estimation of economic impact and waste reduction.

The forecasting model can be improved by incorporating additional variables such as fuel prices, holiday demand, road disruptions, export demand, and real platform transaction data. The user interface can be expanded with Sinhala and Tamil support, voice guidance, and adaptive recommendations based on farmer behavior. The broader FreshRoute platform can further integrate blockchain-based marketplace commitments, freshness-

aware logistics, and AI-based quality verification to create a closed-loop system for transparent and efficient fruit supply chain management in Sri Lanka.

REFERENCES

- [1] L. Rajapaksha, D. M. C. C. Gunathilake, S. M. Pathirana, and T. N. Fernando, "Reducing post-harvest losses in fruits and vegetables for ensuring food security - Case of Sri Lanka," *MOJ Food Processing & Technology*, vol. 9, no. 1, pp. 7-16, 2021, doi: 10.15406/mojft.2021.09.00255.
- [2] S. Kamalakkannan et al., "Life Cycle Assessment of Food Loss Impacts: Case of Banana Postharvest Losses in Sri Lanka," *Procedia CIRP*, vol. 105, pp. 859-864, 2022, doi: 10.1016/j.procir.2022.02.142.
- [3] Ministry of Agriculture, Sri Lanka, "It was revealed that 19% vegetables and 21% fruits waste annually during transportation in Sri Lanka," Mar. 2023. [Online]. Available: <https://www.agrimin.gov.lk/>
- [4] National Institute of Post Harvest Management, "Introduction of safe packaging and transportation methods to reduce postharvest loss in perishable supply chain in Sri Lanka," Jan. 2021. [Online]. Available: <https://www.niphm.lk/>
- [5] L. N. C. De Silva, J. S. Goonetillake, G. N. Wikramanayake, and A. Ginige, "A holistic mobile-based information system to enhance farming activities in Sri Lanka," in *Proc. IASTED Int. Conf. Engineering and Applied Science*, 2012, pp. 91-99.
- [6] M. Aphinaya, R. Rathnayake, S. Sivakumar, and A. M. C. Amarakoon, "Price forecasting of jack fruit using SARIMA model," University of Jaffna, Sri Lanka, 2016.
- [7] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37-45, 2018, doi: 10.1080/00031305.2017.1380080.
- [8] T.-W. Yoo and I.-S. Oh, "Time series forecasting of agricultural products' sales volumes based on seasonal long short-term memory," *Applied Sciences*, vol. 10, no. 22, Art. no. 8169, 2020, doi: 10.3390/app10228169.
- [9] S. Siامي-Namini, N. Tavakoli, and A. Siامي Namin, "A comparison of ARIMA and LSTM in forecasting time series," in *Proc. 17th IEEE Int. Conf. on Machine Learning and Applications*, 2018, pp. 1394-1401, doi: 10.1109/ICMLA.2018.00227.
- [10] H. M. B. P. Ranaweera, R. M. G. H. N. Rathnayake, and A. S. G. J. K. Ananda, "Crop price prediction using machine learning approaches: Reference to the Sri Lankan vegetable market," *Journal of Management Matters*, vol. 10, no. 1, pp. 19-34, 2023.
- [11] P. Wanigasinghe and D. D. M. Ranasinghe, "Price prediction neural network for Econ Centre," in *Proc. 9th Annual Sessions Sri Lanka Association for Artificial Intelligence*, 2012, pp. 1-8.
- [12] M. Fafchamps and B. Minten, "Impact of SMS-based agricultural information on Indian farmers," *World Bank Economic Review*, vol. 26, no. 3, pp. 383-414, 2012.

- [13] Y. H. Gu, Z. L. Yang, X. H. Zhang, and Y. Chen, "Forecasting agricultural commodity prices using dual input attention LSTM," *Agriculture*, vol. 12, no. 2, Art. no. 256, 2022.
- [14] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf.*, 2018, Art. no. 30, pp. 1-15, doi: 10.1145/3190508.3190538.
- [15] A. Kamilaris, A. Fonts, and F. X. Prenafeta-Boldu, "The rise of blockchain technology in agriculture and food supply chains," *Trends in Food Science & Technology*, vol. 91, pp. 640-652, 2019, doi: 10.1016/j.tifs.2019.07.034.
- [16] D. M. Utama, S. K. Dewi, A. Wahid, and I. Santoso, "The vehicle routing problem for perishable goods: A systematic review," *Cogent Engineering*, vol. 7, no. 1, Art. no. 1816148, 2020, doi: 10.1080/23311916.2020.1816148.
- [17] J. Naranjo-Torres et al., "A review of convolutional neural network applied to fruit image processing," *Applied Sciences*, vol. 10, no. 10, Art. no. 3443, 2020, doi: 10.3390/app10103443.
- [18] X. Gao et al., "Application of advanced deep learning models for efficient apple defect detection and quality grading in agricultural production," *Agriculture*, vol. 14, no. 7, Art. no. 1098, 2024, doi: 10.3390/agriculture14071098.
- [19] C. Zauner, "Implementation and benchmarking of perceptual image hash functions," M.S. thesis, Upper Austria University of Applied Sciences, Hagenberg, Austria, 2010.
- [20] M. A. N. S. Perera, P. K. Liyanage, and K. S. S. Perera, "Digital transformation in agriculture: Business models and commercialization strategies for ICT-based solutions in Sri Lanka," *ICTer Journal*, vol. 15, no. 2, pp. 42-56, 2022.
- [21] L. Klerkx, E. Jakku, and P. Labarthe, "A review of social science on digital agriculture, smart farming and agriculture 4.0: New contributions and a future research agenda," *NJAS - Wageningen Journal of Life Sciences*, vol. 90-91, Art. no. 100315, 2019, doi: 10.1016/j.njas.2019.100315.

GLOSSARY

Table G. 1: Glossary of key terms

Term	Definition
Demand Forecasting	The process of predicting future demand or price behavior using historical and contextual data.
LSTM	A recurrent neural network architecture that uses gated memory cells to learn long-term dependencies in sequential data.
Post-Harvest Loss	Loss of quantity or quality after harvesting and before final consumption.
Information Asymmetry	A market condition where one party has more or better information than another, creating unequal bargaining power.
Offline-First Design	A software design approach in which core features remain usable when the device has no active internet connection.
Firebase Cloud Messaging	A cloud messaging service used to send push notifications to mobile devices.
TLS	Transport Layer Security, the modern cryptographic protocol used to secure HTTPS communication.
Redis	An in-memory data store commonly used for caching frequently accessed data.

WAPE	Weighted Absolute Percentage Error, a forecasting metric calculated as total absolute error divided by total actual value.
FreshRoute Price	A platform-calculated indicative price derived from market data and forecasting logic; final formula should be inserted after implementation.

APPENDICES

Appendix A: A Price List at Dambulla Dedicated Economic Center

 දඹුල්ල විශේෂිත ආර්ථික මධ්‍යස්ථාන කළමනාකරණ භාරය දු.අ: -0662285181 Email : dambulladec@gmail.com 					
සංයුක්ත මිල ප්‍රකාශන පොත මිල : (රුපියල්)			දිනය : 2025.08.27		වේලාව : 9.30 am
විස්තරය	මිල පරාසය	මිල පරාසය	විස්තරය	මිල පරාසය	මිල පරාසය
කොළ - Cabbage	160.00	200.00	කොළ මුල් - Lotus Roots	450.00	500.00
කොළ - Cauliflower	250.00	320.00	කොළ මුල් - Kohila	100.00	110.00
රතු කොළ - Red Cabbage			කිරි මුල් - Kiriala	150.00	170.00
ආච්ඡා - Beans	250.00	400.00	මංකු - Manioc	70.00	80.00
ලීක්ස් - Leeks	90.00	120.00	බිත්ත - Sweet Potato	100.00	120.00
කැරට් - කුරුම - Carrot - N	200.00	280.00	කුරුම - Wild - Baby Potato		
කැරට් - ආච්ඡා - Carrot - Jaffna			කුරුම - Onion Leaves	140.00	150.00
කුරුම - කුරුම - Beetroot N	80.00	100.00	කුරුම - Mukunuwenna	20.00	25.00
කුරුම - කුරුම - Beetroot Cut - M	60.00	70.00	කුරුම - Lettuce	200.00	250.00
කුරුම - කුරුම - Beetroot Cut - D	60.00	65.00	කුරුම - Gotukola	20.00	25.00
කුරුම - කුරුම - Beetroot Cut - J	30.00	50.00	කුරුම - Curry Leaves	60.00	70.00
කුරුම - කුරුම - Beetroot Cut - P	70.00	80.00	කුරුම - Athugowa	100.00	150.00
කුරුම - Tomato	120.00	170.00	කුරුම - Spinach	60.00	65.00
රතු - Raddish	30.00	40.00	කුරුම - Water Spinach	20.00	25.00
කොළ - Nnolkhol	60.00	70.00	කුරුම		
කුරුම - Cooking Melon	30.00	40.00	කුරුම - Corns	110.00	120.00
කුරුම - Cucumber	70.00	80.00	කුරුම - Soya Bean	290.00	300.00
කුරුම - කුරුම - Potatoes N	230.00	250.00	කුරුම - Tamarind	490.00	500.00
කුරුම - කුරුම - Potatoes J			කුරුම - Sesame	350.00	500.00
කුරුම - කුරුම - Potatoes W			කුරුම - Green Gram	590.00	600.00
කුරුම - කුරුම - Potatoes Import	170.00	180.00	කුරුම - Cowpea	590.00	600.00
කුරුම - Ridge Gourd	170.00	200.00	කුරුම - Finger Millet	390.00	400.00
කුරුම - Lady's Fingers	70.00	100.00	කුරුම - Gram	350.00	360.00
කුරුම - Snake Gourd	100.00	130.00	කුරුම - Peanuts	490.00	500.00
කුරුම - Bitter Gourd	200.00	220.00	කුරුම		
කුරුම - Thumbakawila	450.00	550.00	කුරුම - Banana - Sini	60.00	70.00
කුරුම - කුරුම - Pumpkin	30.00	35.00	කුරුම - Banana - Ambul	85.00	90.00
කුරුම - කුරුම - Pumpkin	50.00	60.00	කුරුම - Kolikuttu	210.00	220.00
කුරුම - Butternut Squash	50.00	55.00	කුරුම - Ambun Banana	160.00	170.00
කුරුම - Brinjal	300.00	350.00	කුරුම - කුරුම	290.00	300.00
කුරුම - Thithbatu	300.00	350.00	කුරුම - Guava	270.00	280.00
කුරුම - Thai Eggplant	250.00	300.00	කුරුම - Papaya	85.00	90.00
කුරුම - Eggplant	220.00	230.00	කුරුම - Watermelon	85.00	90.00
කුරුම - Winged Bean	200.00	230.00	කුරුම - Pineapple	270.00	280.00
කුරුම - Yard - Long Beans	100.00	200.00	කුරුම - කුරුම	130.00	140.00
කුරුම - Drumsticks	150.00	200.00	කුරුම - කුරුම	540.00	550.00
කුරුම - Ash Plantain	60.00	70.00	කුරුම - TJC	840.00	850.00
කුරුම - Plantain Flower	50.00	60.00	කුරුම - Mango	240.00	250.00
කුරුම - Ambarella	50.00	60.00	කුරුම - Passion	410.00	420.00
කුරුම - Chaw - Chaw	50.00	60.00	කුරුම - Avocado	190.00	200.00
කුරුම - Lime	450.00	550.00	කුරුම - Soursoy	210.00	220.00
කුරුම - Capsicum	200.00	300.00	කුරුම - Gooseberry	1,200.00	1,300.00
කුරුම - Green Chili	120.00	200.00	කුරුම - Woodapple	190.00	200.00
කුරුම - Dry Chilies	580.00	590.00	කුරුම - Indian bael	340.00	350.00
කුරුම - Nai Miris	700.00	1,000.00	කුරුම - King Coconut	120.00	130.00
කුරුම - Genger	850.00	900.00	කුරුම - Coconut	130.00	140.00
කුරුම - Turmeric	240.00	250.00			
කුරුම - Jackfruit	70.00	80.00			
කුරුම - Red Onion Import					
කුරුම - Red Onion - Lanka	130.00	200.00			
කුරුම - B Onion Import	120.00	130.00			
කුරුම - B Onion Lanka					
කුරුම - Garlic	340.00	350.00			

Appendix B: Detailed Test Cases

TC-01: Data preprocessing and feature engineering

Objective: Verify that raw market and weather datasets are cleaned and transformed correctly.

Test steps: Import a dataset containing missing prices, missing rainfall values, outliers, and inconsistent crop names. Run the preprocessing pipeline.

Expected outcome: Missing values are treated, outliers are flagged, crop names are standardized, numerical features are scaled, and sequence windows are generated.

TC-02: LSTM model training and accuracy

Objective: Validate that the model trains successfully and produces acceptable forecasting metrics.

Test steps: Train on chronological training data and evaluate on a held-out test period.

Expected outcome: The script exports MAE, RMSE, MAPE, and WAPE, and stores the trained model artifact.

TC-03: Forecast API response

Objective: Verify that the prediction endpoint returns a valid mobile-ready response.

Test steps: Call POST /predict with crop, market, and horizon parameters.

Expected outcome: API returns valid JSON with prediction values, trend, confidence, recommendation, and model version.

TC-04: Alert threshold triggering

Objective: Ensure alerts are sent when forecast values exceed user thresholds.

Test steps: Set threshold to 10% and generate a forecast with 15% expected increase.

Expected outcome: The alert service records and dispatches SMS and push notifications without duplication.

TC-05: Integration test

Objective: Validate end-to-end flow from data import to mobile display.

Test steps: Import data, run forecast job, retrieve forecast from mobile app, and trigger alert.

Expected outcome: All components exchange consistent identifiers and display correct forecast details.

TC-06: Security test

Objective: Verify protected endpoints and encrypted communication.

Test steps: Attempt unauthenticated access and inspect deployment protocol.

Expected outcome: Protected endpoints reject unauthorized requests and deployment uses HTTPS over TLS.